অধ্যায় ১৭

# JavaScript Regular Expressions, Advanced Validation এবং Secure Input Processing

## এই অধ্যায়ে যা শিখবে

Regular Expression (Regex) ব্যবহার করে Text Pattern খোঁজা, Extract, Replace, Validation, Unicode-aware Processing, Input Normalization এবং নিরাপদ DOM Rendering। অধ্যায়ের শেষে **FormShield** নামে একটি বাস্তব Browser Application তৈরি করবে।

## ১৭.১ Regex কেন প্রয়োজন?

ধরো, একটি Form-এ Email, Phone Number, Full Name এবং Password নেওয়া হচ্ছে। শুধু `value.length > 0` পরীক্ষা করলে ভুল Input আটকানো যায় না। Regex একটি *pattern language*: এটি নির্দিষ্ট Character বা Character Sequence মিলিয়ে দেখতে পারে। তবে Regex কোনো Magic Security Shield নয়—এটি কেবল Validation-এর একটি স্তর।

```javascript
const message = 'Order ID: TRV-2048';
const found = message.match(/TRV-\d{4}/);
console.log(found?.[0]); // TRV-2048
```

`/TRV-\d{4}/`-এ `TRV-` Literal Text, `\d` Digit এবং `{4}` ঠিক চারবার উপস্থিতি বোঝায়।

## ১৭.২ Regex Syntax: Literals, Character Classes এবং Quantifiers

| Pattern | অর্থ | উদাহরণ |
|---|---|---|
| `abc` | সরাসরি `abc` | `abc123` |
| `.` | Line Terminator ছাড়া যেকোনো Character | `a.c` |
| `\d` | ASCII Digit | `2026` |
| `\w` | ASCII Word Character | `user_1` |
| `\s` | Whitespace | Space, Tab |
| `[A-Z]` | Capital English Letter | `HELLO` |
| `[^0-9]` | Digit ছাড়া Character | `a` |
| `?` | ০ অথবা ১ বার | `colou?r` |

| Pattern | অর্থ | উদাহরণ |
|---|---|---|
| `+` | ১ বা একাধিক | `a+` |
| `*` | ০ বা একাধিক | `a*` |
| `{2,5}` | ২ থেকে ৫ বার | `\d{2,5}` |

একটি গুরুত্বপূর্ণ ভুল: `.` মানে Literal Dot নয়। Email Domain-এর Dot মিলাতে `\.` লিখতে হয়।

```javascript
console.log(/a.c/.test('abc'));   // true
console.log(/a\.c/.test('a.c'));  // true
console.log(/a\.c/.test('abc'));  // false
```

## ১৭.৩ Anchors, Groups এবং Alternation

`^` String-এর শুরু, `$` String-এর শেষ বোঝায় (Multiline Flag ছাড়া সাধারণ ব্যবহার)। Validation-এর ক্ষেত্রে Full Input Match করতে এগুলো প্রয়োজন।

```javascript
const codePattern = /^BD-(?:DAC|CGP)-\d{3}$/;
console.log(codePattern.test('BD-DAC-101')); // true
console.log(codePattern.test('XBD-DAC-101')); // false
```

এখানে `(?:...)` Non-capturing Group এবং `|` Alternation। Capturing Group চাইলে `(\d{4})-(\d{2})-(\d{2})` ব্যবহার করা যায়।

```javascript
const date = '2026-10-11';
const parts = date.match(/^(\d{4})-(\d{2})-(\d{2})$/);
console.log(parts?.slice(1)); // ['2026','10','11']
```

**সতর্কতা:** Pattern মিললেই Date বাস্তব হয় না; `2026-99-99`-ও এই Regex-এ মিলে যাবে। পরে Calendar Validation প্রয়োজন।

## ১৭.৪ Regex Flags ও JavaScript Methods

- `i`: Case-insensitive Matching।
- `g`: Global Matches।
- `m`: Multiline Anchors।
- `s`: Dotall Mode।
- `u`: Unicode-aware Mode।

```javascript
const text = 'JS 101, JS 202, JS 303';
console.log([...text.matchAll(/JS (\d+)/g)].map(match => match[1]));
// ['101', '202', '303']
console.log(text.replace(/JS/gi, 'JavaScript'));
console.log('Bangladesh'.search(/desh/i));
```

`test()` Boolean দেয়; `match()` Match Data দেয়; `matchAll()` Iterator দেয়; `replace()` Replacement করে; `split()` Regex Delimiter নিতে পারে। `g` Flag-যুক্ত Regex-এর `test()` বারবার চালালে `lastIndex`-এর কারণে Result বদলাতে পারে—এটি Debugging-এর গুরুত্বপূর্ণ বিষয়।

```javascript
const pattern = /a/g;
console.log(pattern.test('a')); // true
console.log(pattern.test('a')); // false: lastIndex পরিবর্তিত
```

## ১৭.৫ Unicode ও বাংলা Text Validation

বাংলা নাম বা আন্তর্জাতিক নাম Validate করতে শুধু `[A-Za-z]` ব্যবহার করা উচিত নয়। Unicode Property Escape `\p{L}` Letter এবং `\p{M}` Combining Mark বোঝায়। `u` Flag ছাড়া Property Escape প্রত্যাশামতো কাজ করে না।

```javascript
const humanName = /^[\p{L}\p{M}][\p{L}\p{M} .'-]*$/u;
console.log(humanName.test('রহমান'));      // true
console.log(humanName.test('Amina Rahman')); // true
```

বাস্তব নামের বৈচিত্র্য অনেক বেশি। Apostrophe, Hyphen, Script, Transliteration ও Length Requirement ব্যবসায়িক প্রয়োজন অনুযায়ী নির্ধারণ করতে হবে। `String.prototype.normalize('NFC')` Unicode Normalization-এর জন্য কাজে লাগে, তবে এটি সব Linguistic সমস্যা সমাধান করে না।

## ১৭.৬ Normalization বনাম Validation

Input Normalize করার অর্থ হলো অপ্রয়োজনীয় পার্থক্য দূর করা, যেমন Leading/Trailing Space Trim বা Email Lowercase করা। Validation হলো গ্রহণযোগ্যতার নিয়ম পরীক্ষা। দুটিকে আলাদা রাখলে Code Test করা সহজ হয়।

```javascript
function normalizeSpace(value) {
  return String(value ?? '').trim().replace(/\s+/gu, ' ');
}
function validateUsername(value) {
  const username = normalizeSpace(value);
  return /^[a-z][a-z0-9_]{2,19}$/i.test(username);
}
```

কিছু Input কখনো স্বয়ংক্রিয়ভাবে বদলানো উচিত নয়—যেমন Password Trim করলে User-এর দেওয়া Secret পরিবর্তিত হতে পারে।

## ১৭.৭ Email, Phone এবং Password Validation-এর সীমা

Email-এর জন্য একটি সাধারণ Pattern `^[^\s@]+@[^\s@]+\.[^\s@]+$` UI-তে প্রাথমিক Check দিতে পারে। কিন্তু এটি RFC-complete Email Parser নয়। Email সত্যি ব্যবহারযোগ্য কি না জানতে Verification Link বা OTP দরকার।

Phone Number আন্তর্জাতিকভাবে জটিল; Production-এ Country-aware Phone Parsing Library প্রয়োজন হতে পারে। Password-এর ক্ষেত্রে শুধু Regex Complexity Rule যথেষ্ট নয়। Server-এ Secure Password Hashing, Rate Limiting, Breached Password Check এবং MFA গুরুত্বপূর্ণ।

```javascript
function basicEmailCheck(value) {
  const email = String(value).trim();
  return email.length <= 254 && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

## ১৭.৮ Regex Security: ReDoS ও Untrusted Input

Nested Quantifier-এর মতো কিছু Pattern-এ Catastrophic Backtracking হতে পারে। উদাহরণস্বরূপ, `^(a+)+$` দীর্ঘ `aaaa...!` Input-এ অত্যন্ত ধীর হতে পারে। তাই Untrusted Text-এর ক্ষেত্রে Regex-এর Complexity, Maximum Input Length এবং বিকল্প Parser বিবেচনা করো।

**আরেকটি ঝুঁকি:** User Input সরাসরি `new RegExp(userInput)`-এ বসালে Special Character Pattern বদলে দিতে পারে। Literal Search চাইলে Escape করো।

```javascript
function escapeRegex(value) {
  return String(value).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}
const query = 'a+b';
const safePattern = new RegExp(escapeRegex(query), 'i');
console.log(safePattern.test('Search: a+b')); // true
```

## ১৭.৯ XSS, Injection এবং Secure DOM Rendering

Regex দিয়ে HTML Sanitization করার চেষ্টা করো না। Untrusted Text দেখাতে `textContent` ব্যবহার করো। `innerHTML` দিয়ে User-supplied Content বসানো বিপজ্জনক হতে পারে। HTML প্রয়োজন হলে নির্ভরযোগ্য Sanitization Library ও Strict Content Security Policy বিবেচনা করো।

```javascript
const display = document.querySelector('#output');
display.textContent = '<img src=x onerror=alert(1)>';
// Text হিসেবে দেখাবে; HTML Execute করবে না।
```

SQL Injection Regex দিয়ে প্রতিরোধ করা যায় না; Database-এ Parameterized Query ব্যবহার করতে হয়। Client-side Validation সহজেই Bypass করা যায়, তাই Server-side Validation বাধ্যতামূলক।

## ১৭.১০ Practical Project — FormShield

**Project Goal:** User-এর Full Name, Email, Phone, Password এবং Consent যাচাই করে পরিষ্কার Error Feedback দেখানো। এটি শুধুই Browser-side Demonstration; কোনো Personal Data Network-এ পাঠায় না বা Storage-এ রাখে না।

### Folder Structure

```
source-code/chapter-17/
├── index.html
├── style.css
├── form-core.mjs
├── app.mjs
└── form-core.test.mjs
```

`form-core.mjs`-এ Pure Validation Functions, `app.mjs`-এ Form Events এবং DOM Update, `index.html`-এ Accessible Form এবং `style.css`-এ Responsive Layout আছে। পূর্ণ Runnable Source Code বইয়ের Companion Source Code-এর `source-code/chapter-17/` Folder-এ সংযুক্ত।

### Core Validation Flow

JAVASCRIPT

```javascript
import { validateRegistration } from './form-core.mjs';
const result = validateRegistration({
  name: 'Amina Rahman',
  email: 'amina@example.com',
  phone: '+8801712345678',
  password: 'SecurePass1234',
  consent: true
});
console.log(result.ok); // true
```

### Run Instructions

1. Node.js 20+ ইনস্টল করো।
2. `node --test source-code/chapter-17/form-core.test.mjs` দিয়ে Tests চালাও।
3. Browser Module চালানোর জন্য Folder Root-এ `python -m http.server 8000` দিয়ে Local Server চালাতে পারো।
4. `http://localhost:8000/source-code/chapter-17/` খুলে Form পরীক্ষা করো।
5. Invalid Email, Bengali Name, Long Input, Missing Consent এবং HTML Injection Text দিয়ে Test করো।

### Project Extension Ideas

Field-level Live Validation, Accessible Error Summary, Country-aware Phone Parser, Password Strength Feedback এবং Backend Validation Endpoint যুক্ত করতে পারো। Password কখনো Console-এ Log বা Browser Storage-এ Save করবে না।

## ১৭.১১ Exercises

**Exercise 1:** `BD-1234` Format Validate করার Regex লেখো।

**Exercise 2:** একটি String থেকে সব ৪-অঙ্কের Year বের করো।

**Exercise 3:** `test()`-এর Global `lastIndex` Pitfall প্রদর্শন করো।

**Exercise 4:** বাংলা ও ইংরেজি Letter গ্রহণ করে এমন Name Pattern তৈরি করো।

**Exercise 5:** Email Normalize এবং Validate করার দুটি আলাদা Function লেখো।

**Exercise 6:** User Search Query-কে Regex Literal হিসেবে Escape করো।

**Exercise 7:** Phone Number থেকে Space ও Hyphen সরিয়ে Digit Count Check করো।

**Exercise 8:** `innerHTML`-এর বদলে `textContent` ব্যবহার করে Safe Preview বানাও।

**Exercise 9:** Invalid Date Pattern এবং Invalid Calendar Date-এর পার্থক্য বোঝাও।

**Exercise 10:** FormShield-এ Password Confirmation Field যুক্ত করো।

## ১৭.১২ Sample Solutions

```javascript
// 1
const codeOK = /^BD-\d{4}$/.test('BD-1234');
// 2
const years = [...'1999, 2026 and 2040'.matchAll(/\b\d{4}\b/g)].map(m => m[0]);
// 3
const global = /x/g; console.log(global.test('x'), global.test('x'));
// 4
const nameOK = /^[\p{L}\p{M} ]+$/u.test('রহমান Ali');
// 5
const normalizeEmail = v => String(v).trim().toLowerCase();
const emailOK = v => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalizeEmail(v));
// 6
const escapeRegex = v => String(v).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
// 7
const digits = '+880 1712-345678'.replace(/[\s-]/g, '');
const phoneOK = /^\+?\d{7,15}$/.test(digits);
// 8 (Browser)
// document.querySelector('#preview').textContent = userInput;
// 9: 2026-99-99 Regex match করতে পারে, কিন্তু Calendar Date নয়।
// 10: password === confirmation পরীক্ষা করো; Password Log/Store কোরো না।
```

## ১৭.১৩ MCQ Quiz

১. Regex-এ `\d` কী? A) Letter B) Digit C) Dot D) Space

২. `^` ও `$` একসঙ্গে কেন? A) Style B) Full Input Match C) API Call D) JSON

৩. বাংলা Unicode Property Escape-এর জন্য কোন Flag? A) g B) i C) u D) m

৪. `test()` কী Return করে? A) Boolean B) Array C) Promise D) Object

৫. `g` Flag-সহ Repeated `test()`-এ কোন Property বদলায়? A) length B) lastIndex C) prototype D) size

৬. Regex দিয়ে HTML Sanitization কি নিরাপদ? A) হ্যাঁ B) না C) শুধু Chrome D) শুধু Node

৭. Untrusted Text দেখানোর নিরাপদ DOM Property কোনটি? A) innerHTML B) outerHTML C) textContent D) insertAdjacentHTML

৮. ReDoS সাধারণত কোন সমস্যার সঙ্গে যুক্ত? A) CSS B) Catastrophic Backtracking C) URL Encoding D) JSON

৯. Client-side Validation কি Server-side Validation-এর বিকল্প? A) হ্যাঁ B) না C) শুধু HTTPS D) শুধু SPA

১০. Email Regex Pass করলে কি Inbox Ownership নিশ্চিত হয়? A) হ্যাঁ B) না C) শুধু Gmail D) শুধু Corporate

### Answer Key ও Explanation

1. **B** — `\d` Digit Class।
2. **B** — Start ও End Anchor দিয়ে সম্পূর্ণ Input Check করা যায়।
3. **C** — `u` Unicode Mode চালু করে।
4. **A** — `test()` True বা False দেয়।
5. **B** — Global/Sticky Regex-এর `lastIndex` Stateful।
6. **B** — HTML Sanitization-এর জন্য Regex নির্ভরযোগ্য নয়।
7. **C** — `textContent` Markup Execute করে না।
8. **B** — Complex Backtracking CPU Time বাড়াতে পারে।
9. **B** — Client Validation Bypass করা যায়।
10. **B** — Ownership Verify করতে Confirmation দরকার।

## ১৭.১৪ Chapter Recap

Regex একটি শক্তিশালী Text Processing Tool, কিন্তু Validation, Security ও Business Logic আলাদা বিষয়। Anchors, Groups, Quantifiers, Unicode, `matchAll()`, Normalization, ReDoS Awareness এবং Safe DOM Rendering—এই দক্ষতাগুলো FormShield Project-এ একসঙ্গে প্রয়োগ করেছ। পরবর্তী অধ্যায়ে Functional Programming, Immutability এবং Advanced Data Transformation নিয়ে কাজ করব।